LeetCode 890: Find and Replace Pattern

1. Problem Title & Link

Title: LeetCode 890: Find and Replace Pattern

Link: https://leetcode.com/problems/find-and-replace-pattern/

2. Problem Statement (Short Summary)

You are given:

- A list of words
- A pattern string

Return all words that match the pattern.

A word matches if there exists a **one-to-one mapping** between characters in the pattern and characters in the word.

This means:

One pattern character

→ maps to exactly one word character

One word character

→ maps to exactly one pattern character

(Bijective Mapping)

3. Examples (Input → Output)

Example 1

Input:

words = ["abc","deq","mee","aqq","dkd","ccc"]

pattern = "abb"

Output:

["mee","aqq"]

Explanation:

Pattern:

a → m

b → e

Word:

mee

Valid.

Example 2

Input:
words = ["a","b","c"]

pattern = "a"
Output:
["a","b","c"]

Example 3
Input:
words = ["xyz","xyy","aba"]

pattern = "abb"
Output:
["xyy"]

4. Constraints

- 1 ≤ words.length ≤ 50
- 1 ≤ pattern.length ≤ 20
- All words have same length as pattern
- Lowercase English letters only

5. Core Concept (Pattern / Topic)
HashMap / Character Mapping
Related Concepts:

- Bijective Mapping
- Isomorphic Strings
- Hashing

6. Thought Process (Step-by-Step Explanation)
Brute Force
For every word:
Try all possible character mappings.
Very complicated and unnecessary.

Key Observation
Pattern:
abb

Word:

mee

Mapping:

a → m

b → e

Valid because:

Same pattern chars

→ same word chars

Different pattern chars

→ different word chars

Need Two-Way Mapping

Using only:

pattern → word

is not enough.

Example:

pattern = "ab"

word = "aa"

Mapping:

a → a

b → a

Invalid.

Two pattern characters cannot map to same character.

Need:

pattern → word

word → pattern

Why It Works

Two hashmaps guarantee:

One-to-one mapping

which is exactly what the problem requires.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Pattern:
a b b
Word:
m e e
Mappings:
a → m

b → e
Reverse:
m → a

e → b
Both consistent.
Valid.
```

8. Pseudocode (Language Independent)

```
result = []

for each word:

    create map1
    create map2

    valid = true

    for each character:

        verify mapping

    if valid:
        add word

return result
```

9. Code Implementation

✅ **Python**

```
class Solution:

    def findAndReplacePattern(
        self,
        words,
        pattern
    ):

        def matches(word):
```

```
            p_to_w = {}
            w_to_p = {}

            for p, w in zip(pattern, word):

                if p in p_to_w:

                    if p_to_w[p] != w:
                        return False

                else:
                    p_to_w[p] = w

                if w in w_to_p:

                    if w_to_p[w] != p:
                        return False

                else:
                    w_to_p[w] = p

            return True

        result = []

        for word in words:

            if matches(word):
                result.append(word)

        return result
```

✅ **Java**

```
import java.util.*;

class Solution {

    public List<String> findAndReplacePattern(
        String[] words,
        String pattern
    ) {

        List<String> answer =
            new ArrayList<>();

        for (String word : words) {

            if (matches(word, pattern))
```

```
                answer.add(word);
            }

            return answer;
        }

        private boolean matches(
            String word,
            String pattern
        ) {

            Map<Character, Character>
                pToW = new HashMap<>();

            Map<Character, Character>
                wToP = new HashMap<>();

            for (int i = 0;
                 i < word.length();
                 i++) {

                char p = pattern.charAt(i);
                char w = word.charAt(i);

                if (pToW.containsKey(p) &&
                    pToW.get(p) != w)
                    return false;

                if (wToP.containsKey(w) &&
                    wToP.get(w) != p)
                    return false;

                pToW.put(p, w);
                wToP.put(w, p);
            }

            return true;
        }
    }
```

10. Time & Space Complexity

Time Complexity

Let:

n = number of words

m = length of pattern

For every word:

O(m)

Total:

O(n × m)

Space Complexity

O(m)

For hashmaps.

11. Common Mistakes / Edge Cases

Mistake 1

Using only one hashmap.

Example:

pattern = "ab"

word = "aa"

Incorrectly becomes valid.

Mistake 2

Checking frequency only.

Example:

abb

bcc

Frequency matches.

Need mapping validation too.

Mistake 3

Not checking reverse mapping.

12. Detailed Dry Run (Step-by-Step Table)

Input:

word = "mee"

pattern = "abb"

Index	Pattern	Word	p→w	w→p
0	a	m	a→m	m→a

1	b	e	b→e	e→b
2	b	e	valid	valid

Result:

True

Example:

word = "ccc"

pattern = "abb"

Index	Pattern	Word
0	a	c
1	b	c

Now:

a → c

b → c

Invalid.

Return:

False

13. Common Use Cases (Real-Life / Interview)

Data Encoding

Map symbols between formats.

Language Translation

Character substitution systems.

Compiler Design

Variable renaming consistency.

Pattern Recognition

Detect structural similarity.

14. Common Traps (Important!)

- Using one hashmap only
- Comparing frequencies instead of mappings

- Forgetting reverse mapping
- Confusing isomorphic vs anagram problems

15. Builds To (Related LeetCode Problems)

1. LC 205 — Isomorphic Strings
2. LC 290 — Word Pattern
3. LC 49 — Group Anagrams
4. LC 242 — Valid Anagram
5. LC 890 — Find and Replace Pattern

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force Mapping	Large	Large	Poor
Frequency Check	Incorrect	O(1)	Fails
Two HashMaps	O(n×m)	O(m)	Optimal

17. Why This Solution Works (Short Intuition)

The problem requires a bijection (one-to-one correspondence). Two hashmaps ensure that every pattern character maps to exactly one word character and every word character maps back to exactly one pattern character.

18. Variations / Follow-Up Questions

Follow-Up 1

What if words have different lengths?

Immediately reject lengths different from pattern.

Follow-Up 2

What if uppercase letters are included?

No change in logic.

Follow-Up 3

What if pattern contains words instead of characters?

Use:

split()

and apply the same mapping logic